Test Task: MSI Product Page Scraper

Junior / Trainee JavaScript Developer

Goal	Build a small production-style scraper for one MSI product detail page.
Estimated time	2-3 hours. Please do not over-engineer the solution.
Required stack	Node.js, JavaScript, Playwright.
Deliverable	A public GitHub repository with source code, package.json, and sample JSON output.

1. Background

This task simulates a real web data extraction assignment. Your goal is to inspect the page, identify reliable selectors or page data sources, and return a clean JSON object that follows the schema below. The solution should be simple, readable, and easy to run locally.

2. Target page

Scrape the following product detail page:

https://us-store.msi.com/Motherboards/Intel-Platform-Motherboard/INTEL-Z890/MAG-Z890-TOMAHAWK-WIFI

3. Task requirements

- Use Playwright with JavaScript. TypeScript is optional, but plain JavaScript is fully acceptable.
- Open the product page, wait for the required content, and extract product data from the live page.
- Use robust selectors or data extraction logic. Avoid relying only on fragile absolute selectors.
- Normalize prices to numbers, for example "$299.99" should become 299.99.
- Normalize missing fields as null, and use empty arrays only for list fields when nothing is found.
- Save the result to output/product.json.
- Do not hardcode the final product data. The target URL may be a constant, but values should be read from the page.
- Do not commit node_modules, browser binaries, secrets, or local environment files.

4. Expected simplified schema

Return one JSON object with the fields below. Keep the field names exactly as specified.

Field	Type	Notes
url	string	Final product page URL.
item_id	string | null	Product ID from page data, URL, HTML, or null if not available.
title	string | null	Product title.
brand	string | null	Expected value is usually MSI.
product_category	string | null	Readable category path, for example breadcrumb text joined with >.

category_tree	array	Breadcrumb entries: name and URL where available.
description	string | null	Main product description or summary.
price	number | null	Regular price as a number.
sale_price	number | null	Discounted price if visible; otherwise null.
availability	string | null	Use in_stock, out_of_stock, pre_order, or null.
image_url	string | null	Main product image URL.
additional_image_urls	array	Additional product image URLs, excluding duplicates when possible.
specs	array	Key-value technical specifications found on the page.
star_rating	number | null	Average rating if visible.
review_count	number | null	Number of reviews if visible.
gtin	string | null	GTIN/UPC if visible or available in page data.
mpn	string | null	Manufacturer part number if visible or available in page data.
scraped_at	string	Current datetime in ISO 8601 format.

5. JSON shape

```
{
  "url": "string",
  "item_id": "string | null",
  "title": "string | null",
  "brand": "string | null",
  "product_category": "string | null",
  "category_tree": [
    {
      "name": "string",
      "url": "string | null"
    }
  ],
  "description": "string | null",
  "price": "number | null",
  "sale_price": "number | null",
  "availability": "\"in_stock\" | \"out_of_stock\" | \"pre_order\" | null",
  "image_url": "string | null",
  "additional_image_urls": [
    "string"
  ],
  "specs": [
    {
      "name": "string",
      "value": "string | null"
    }
  ],
  "star_rating": "number | null",
  "review_count": "number | null",
  "gtin": "string | null",
  "mpn": "string | null",
  "scraped_at": "ISO 8601 datetime string"
}
```

6. Recommended project structure

```
msi-product-scraper/
  package.json
  src/
```

```
    scrape.js
  output/
    product.json
```

7. Example CLI behavior

The reviewer should be able to install dependencies and run the scraper with commands similar to:

```
npm install
npx playwright install chromium
npm run scrape
```

After running the command, output/product.json should be created or overwritten.

8. Sample output format

Values below are examples only. Your scraper should extract current values from the page at runtime.

```
{
  "url": "https://us-store.msi.com/Motherboards/Intel-Platform-Motherboard/INTEL-Z890/MAG-Z890-TOMAHAWK-
WIFI",
  "item_id": null,
  "title": "MAG Z890 TOMAHAWK WIFI",
  "brand": "MSI",
  "product_category": "Motherboards > Intel Platform Motherboard > INTEL Z890",
  "category_tree": [
    {
      "name": "Motherboards",
      "url": "https://us-store.msi.com/Motherboards"
    },
    {
      "name": "Intel Platform Motherboard",
      "url": null
    },
    {
      "name": "INTEL Z890",
      "url": null
    }
  ],
  "description": null,
  "price": 299.99,
  "sale_price": null,
  "availability": "in_stock",
  "image_url": "https://example.com/main-image.png",
  "additional_image_urls": [
    "https://example.com/image-2.png"
  ],
  "specs": [
    {
      "name": "Chipset",
      "value": "Intel Z890"
    },
    {
      "name": "CPU Support",
      "value": "Intel Core Ultra processors"
    }
  ],
  "star_rating": null,
  "review_count": null,
  "gtin": null,
  "mpn": null,
```

```
  "scraped_at": "2026-06-17T10:30:00.000Z"
}
```

9. Acceptance criteria

- The project runs successfully from a clean install using the commands in this task description.
- The scraper uses Playwright and works in headless mode.
- The output is valid JSON and matches the simplified schema.
- At least title, brand, price or sale_price, availability, image_url, and several specs are extracted when available on the page.
- The code is readable, split into small helper functions where useful, and includes basic error handling.
- The implementation avoids unnecessary complexity and stays within the 2-3 hour timebox.

10. Evaluation focus

Area	What we look for
Page analysis	Can you inspect the DOM and page behavior using DevTools?
Selector quality	Are selectors reasonably stable and understandable?
Data normalization	Are prices, empty values, arrays, and strings normalized consistently?
Code quality	Is the solution easy to read, run, and maintain?
Practical judgment	Did you solve the task without over-engineering it?

11. Submission instructions

1. Create a GitHub repository for the completed test task.
2. Make the repository public so the reviewer can access it without requesting permissions.
3. Commit the full solution: source code, package.json, and output/product.json.
4. When you finish, share the public GitHub repository link with the recruiter or hiring team.

12. Notes

- You may use CSS selectors, XPath, or page scripts where appropriate.
- You may inspect network requests if the page exposes useful structured data.
- If a field is not available on the page, set it to null instead of guessing.
- Keep the solution focused on this single product page.